

Optic-SpaceNet 光计算迁移实验记录

三个 CNN 模型向 int4 光计算硬件 (8×2 光学矩阵乘法器) 迁移的完整实验轨迹

目录

- [实验目标](#)
- [模型与基准](#)
- [实验路线图](#)
- [Phase 0: FP32 基准训练](#)
- [Phase 1: QAT 微调 \(FP32→int4\)](#)
- [Phase 2: 从零 QAT 训练 \(动态 scale\)](#)
- [Phase 3: LSQ + 混合精度 \(当前\)](#)
- [关键 Bug 记录](#)
- [文件清单](#)
- [结论与下一步](#)

1. 实验目标

将三个在 EuroSAT (10 类遥感图像) 上训练的 CNN 模型迁移到 int4 精度光计算硬件上运行，在保持硬件高利用率 (>95%) 的前提下，最小化 int4 量化带来的精度损失。

光计算硬件约束

参数	规格
矩阵乘法器	8×2 向量单元
权重精度	int4 (对称, [-8, 7])
输入精度	int4
对齐要求	im2col 展平长度被 8 整除

精度目标

模型	FP32 准确率	int4 目标准确率	允许损失
Model 1 (VGG)	97.17%	$\geq 95\%$	$\leq 2\%$
Model 2 (SN V1)	90.15%	$\geq 86\%$	$\leq 4\%$
Model 3 (SN V2 KD)	91.44%	$\geq 87\%$	$\leq 4\%$

2. 模型与基准

2.1 三个模型

	Model 1	Model 2	Model 3
名称	Baseline VGG	OpticSpaceNet V1	OpticSpaceNet V2
架构	Mini-VGG, 全 3×3	硬件对齐 CNN	硬件对齐 CNN
参数量	2,386,986	268,210	268,210
训练方式	标准分类	标准分类	知识蒸馏 (KD)
教师模型	—	—	ResNet-18 (97.83%)
硬件对齐率	~99.8%	~96.3%	~96.3%
未对齐层	block1.0 (27→32)	stem.0 (3→8)	stem.0 (3→8)

2.2 FP32 训练配置 (共用)

参数	值
数据集	EuroSAT RGB (27,000 张, 64×64)
划分	80% 训练 / 20% 验证 (seed=42)
Batch size	64
优化器	Adam
学习率	0.001
调度器	CosineAnnealingLR
数据增强	RandomHorizontalFlip, RandomRotation(10°)
归一化	ImageNet mean/std

3. 实验路线图

Phase 0: FP32 基准训练

- Model 1: 97.17% ✓
- Model 2: 90.15% ✓
- Model 3: 91.44% ✓ (KD, 教师 97.83%)

Phase 1: QAT 微调 (FP32 权重 → int4 QAT fine-tuning)

- Model 1: 85.91% ✗ (损失 -11.26%)
- Model 2: 73.63% ✗ (损失 -16.52%)
- Model 3: 73.22% ✗ (损失 -18.22%)
- 结论: FP32→int4 微调路线失败, 模型被困在坏的局部最小值

Phase 2: 从零 QAT 训练 (随机初始化 + int4 从 epoch 1)

- Model 1: 91.17% △ (损失 -6.00%, 过拟合)
- Model 2: 81.20% △ (损失 -8.95%, 欠拟合)
- Model 3: 83.26% △ (损失 -8.18%, KD 有帮助)
- 结论: 优于微调但仍有差距, 需优化训练策略

Phase 3: LSQ + 混合精度 (当前)

- 权重量化: LSQ 可学习 scale (从统计初始化, 梯度优化)
- 输入量化: 动态 scale (适应快速变化的激活分布)
- 混合精度: 首层 + 末层 float32, 其余 int4
- 状态: 训练中 (修复 LSQ 梯度 shape bug)

4. Phase 0: FP32 基准训练

Model 1: Baseline VGG

指标	值
脚本	model1_baseline.py
权重	baseline_vgg.pth (9.1 MB)
Epochs	60
耗时	112.6 分钟 (CPU)
最佳准确率	97.17%
对齐率	99.8% (首层 84.4%, 其余 100%)

关键观察:

- 训练收敛快: epoch 10 已达 92.56%
- 轻微过拟合: train 99.67% vs val 97.07%

Model 2: SpaceNet V1

指标	值
脚本	model2_spacenet_v1.py
权重	spacenet_v1.pth (1.0 MB)
Epochs	80
耗时	45.2 分钟 (CPU)
最佳准确率	90.15%
对齐率	96.3% (stem 37.5%, 其余 100%)

关键观察:

- 小模型收敛慢但稳定
- 无过拟合: train 89.12% vs val 89.98%

Model 3: SpaceNet V2 (KD)

指标	值
脚本	model3_spacenet_v2.py
学生权重	spacenet_v2_distilled.pth (1.0 MB)
教师权重	teacher_resnet18.pth (42.7 MB)
教师 Epochs	30
学生 Epochs	100
教师最佳	97.83%
学生最佳	91.44%
KD 超参数	T=4.0, $\alpha=0.5$

关键观察:

- KD 比独立训练提升约 1.3% (90.15% → 91.44%)
 - 教师计算量大但在可接受范围
-

5. Phase 1: QAT 微调 (FP32→int4)

方案说明

加载 FP32 预训练权重 → BN 融合到 Conv → 替换为 QAT 层 → 低学习率 (1e-4) 微调 15-20 epochs。

5.1 Model 1 QAT 微调

实验	状态	Val Acc	Gap	说明
第 1 次	Bug	91.61%	0.00% (bug)	eval 时未施加量化 (Bug #1)
第 2 次	✓	85.91%	+2.94%	Bug 修复后, QAT 模式 85.91%, Float 88.85%

结论: FP32 97.17% → QAT 85.91%，损失 **-11.26%**。大模型有回旋余地但远不够。

5.2 Model 2 QAT 微调

实验	状态	Val Acc	Gap	说明
第 1 次	Bug	63.24%	0.00% (bug)	eval 时未施加量化
第 2 次	✓	73.63%	-16.41%	QAT 73.63%, Float 57.22%

结论: FP32 90.15% → QAT 73.63%，损失 **-16.52%**。小模型 FP32 权重被 BN 融合 + int4 量化完全破坏，无法恢复。

5.3 Model 3 QAT 微调

实验	状态	Val Acc	Gap	说明
第 1 次	Bug	—	—	import numpy as np 缺失 (Bug #2)
第 2 次	✓	73.22%	-9.09%	KD+QAT 联合微调, QAT 73.22%, Float 64.13%

结论: FP32 91.44% → QAT 73.22%，损失 **-18.22%**。KD 权重同样被量化破坏。

5.4 Phase 1 总结

模型	FP32	QAT 微调	损失	判定
Model 1	97.17%	85.91%	-11.26%	✗ 失败
Model 2	90.15%	73.63%	-16.52%	✗ 灾难

模型	FP32	QAT 微调	损失	判定
Model 3	91.44%	73.22%	-18.22%	✗ 灾难

根因: FP32 权重学到的特征依赖 float32 精细精度，突然被 int4 量化后特征被破坏。
15-20 个 epoch + 低学习率无法让模型从坏的局部最小值中逃逸。

6. Phase 2: 从零 QAT 训练 (动态 scale)

方案说明

随机初始化 → 全程 int4 伪量化 (epoch 1 起) → 模型从未见过 float32 → 特征天然兼容 int4。

与 Phase 1 的关键区别:

- 不加载 FP32 预训练权重
- 保留 BN 层 (不融合) — 稳定训练
- 标准学习率 0.001 (与 FP32 训练相同)
- 标准训练轮数 (60/80/100)

6.1 Model 1 从零 QAT

指标	值
脚本	model1_baseline_int4.py
Epochs	60
Scale 方式	动态 (每步 max/7)
最佳准确率	91.17%
Float32 模式	84.24%
Train acc	95.39% (过拟合 4.22%)

关键观察:

- 相比微调提升 +5.26% (85.91% → 91.17%)
- 明显过拟合: train 95.39% vs val 91.17%
- Int4 > Float: 权重专为 int4 优化

6.2 Model 2 从零 QAT

指标	值
脚本	model2_spacenet_v1_int4.py
Epochs	80
Scale 方式	动态
最佳准确率	81.20%
Float32 模式	69.17%

关键观察:

- 相比微调提升 +7.57% (73.63% → 81.20%)
- 收敛慢: 45 epochs 才突破 80%
- 欠拟合: train 80.95% ≈ val 81.20%

6.3 Model 3 从零 KD+QAT

指标	值
脚本	model3_spacenet_v2_int4.py
Epochs	100
最佳准确率	83.26%
Float32 模式	67.37%

关键观察:

- 相比微调提升 +10.04% (73.22% → 83.26%)
- KD 从零引导有效: 比 Model 2 高 +2.06%
- 收敛极慢: 90 epochs 才达峰值

6.4 Phase 2 总结

模型	FP32	QAT 微调	从零 QAT	提升	与 FP32 差距
Model 1	97.17%	85.91%	91.17%	+5.26%	-6.00%
Model 2	90.15%	73.63%	81.20%	+7.57%	-8.95%
Model 3	91.44%	73.23%	83.26%	+10.03%	-8.18%

进步: 从零 QAT 比微调方案有 5-10% 的绝对提升，验证了"从零学习 int4 兼容特征"策略。

不足: 仍比 FP32 低 6-9%，主要问题:

- 1. 动态 scale ($\max/7$) 每 batch 波动大，量化目标不稳定
- 2. 首层 RGB→int4 和末层 logits→int4 损失大
- 3. Model 1 明显过拟合，Model 2/3 收敛缓慢

7. Phase 3: LSQ + 混合精度 (当前)

方案说明

在 Phase 2 基础上的三点关键优化:

优化 1: LSQ 可学习权重 scale

- 每层 QAT 有 `weight_scale` 参数 (`nn.Parameter`)
- 从 weight 统计初始化: `scale = max(|W_per_channel|) / 7`
- 通过 LSQ 梯度公式优化: 模型自己学最优量化范围
- 梯度缩放: $1/\sqrt{N * n_levels}$, 确保不同大小层梯度一致

优化 2: 输入动态 scale (保留)

- 输入激活分布训练中快速变化，动态 scale 更稳定
- 不需要学习，避免 LSQ 初始化困难 (Phase 3 第一次尝试的教训)

优化 3: 混合精度

- 首层 Conv (RGB 输入) → float32 (3 通道 int4 信息损失太大)
- 末层 Linear (分类 logits) → float32 (直接决定预测)
- 中间层 → int4 QAT
- 首/末层参数占比 < 0.1%，对光计算效率影响可忽略

训练配置

	Model 1	Model 2	Model 3
脚本	<code>model1_baseline_int4.py</code>	<code>model2_spacenet_v1_int4.py</code>	<code>model3_spacenet_v</code>
Epochs	60	100	120
LR	0.001	0.001	0.001
Weight decay	5e-4	1e-4	1e-4

	Model 1	Model 2	Model 3
Float32 层	block1.0 + classifier.4	stem.0 + classifier.4	stem.0 + classifier.4
输入 scale	动态	动态	动态
权重 scale	LSQ 可学习	LSQ 可学习	LSQ 可学习

7.1 Model 1 首次运行 (已修复)

指标	值
问题	LSQ input_scale 初始化为 1.0，内部层激活量化到仅 3-4 级别
现象	Epoch 1-40: loss 不降, acc ≈ 11% (随机猜测)
根因	input_scale 用 LSQ 学习但初始化错误，调不动
修复	输入改回动态 scale，仅权重用 LSQ
状态	待重新训练

7.2 Phase 3 当前状态

模型	状态	Roadblock
Model 1	代码就绪，待跑	LSQ input_scale bug 已修复
Model 2	代码就绪，待跑	同上
Model 3	代码就绪，待跑	同上

8. 关键 Bug 记录

Bug #1: QAT eval 模式未施加量化 (已修复)

文件: optic_qat.py , QATConv2d.forward / QATLinear.forward

原代码:

```
if self._qat_enabled and self.training: # ← BUG
```

问题: `model.eval()` 时 `self.training=False`, 量化被跳过。

训练用 int4 但验证用 float32 → 测不准 → gap 永远 0%。

修复:

```
if self._qat_enabled: # 不管 train/eval, QAT 开着就量化
```

Bug #2: Model 3 QAT 脚本缺少 import (已修复)

文件: `model3_spacenet_v2_qat.py`

问题: `import numpy as np` 缺失, `load_data()` 中 `np.random.RandomState(SEED)` 报 `NameError`。

Bug #3: LSQ gradient shape 不匹配 (已修复)

文件: `optic_qat.py`, `_LSQInt4Fn.backward`

原代码:

```
grad_scale = grad_scale.sum() / (N * n_levels) ** 0.5 # 标量
```

问题: `weight_scale` 参数形状是 `(C_out, 1, 1, 1)`, 但 `backward` 返回标量 → `shape mismatch`。

修复: 沿 broadcast 维度 sum, 保持 `scale.shape` 一致:

```
sum_dims = [d for d in range(x.dim())
             if d >= scale.dim() or scale.shape[d] == 1]
if sum_dims:
    grad_scale = grad_scale.sum(dim=sum_dims)
grad_scale = grad_scale.view(scale.shape)
grad_scale = grad_scale / (N * n_levels) ** 0.5
```

Bug #4: LSQ input_scale 初始化导致训练停滞 (已修复)

文件: `optic_qat.py`, `QATConv2d` / `QATLinear`

问题: `input_scale` 初始化为 1.0, 但内部层激活值分布未知。LSQ 梯度太小, `scale` 调整不动 → 输入被量化为 3-4 个级别 → 梯度消失 → 模型不学习。

修复: 输入量化使用动态 `fake_int4_quantize` (每步计算 `scale`), 仅权重使用 LSQ。

9. 文件清单

训练脚本

文件	行数	用途	产出权重
<code>model1_baseline.py</code>	289	Model 1 FP32 标准训练	<code>baseline_vgg.pth</code>
<code>model2_spacenet_v1.py</code>	293	Model 2 FP32 标准训练	<code>spacenet_v1.pth</code>
<code>model3_spacenet_v2.py</code>	434	Model 3 FP32 KD 训练	<code>spacenet_v2_distilled.pth</code> , <code>teacher_resnet18.pth</code>
<code>model1_baseline_qat.py</code>	333	Model 1 QAT 微调 (Phase 1)	<code>baseline_vgg_qat.pth</code>
<code>model2_spacenet_v1_qat.py</code>	337	Model 2 QAT 微调 (Phase 1)	<code>spacenet_v1_qat.pth</code>
<code>model3_spacenet_v2_qat.py</code>	442	Model 3 QAT 微调 (Phase 1)	<code>spacenet_v2_qat.pth</code>
<code>model1_baseline_int4.py</code>	335	Model 1 从零 QAT (Phase 2/3)	<code>baseline_vgg_int4.pth</code>
<code>model2_spacenet_v1_int4.py</code>	360	Model 2 从零 QAT (Phase 2/3)	<code>spacenet_v1_int4.pth</code>
<code>model3_spacenet_v2_int4.py</code>	362	Model 3 从零 KD+QAT (Phase 2/3)	<code>spacenet_v2_int4.pth</code>

核心库

文件	行数	用途
<code>optic_qat.py</code>	860	QAT 核心: <code>fake_int4_quantize</code> , <code>lsq_int4_quantize</code> , <code>QATConv2d</code> , <code>QATLinear</code> , <code>prepare_qat_model*</code> , BN 融合, 校准, 评估

文件	行数	用途
optic_layers.py	891	光计算推理: OpticalEngine , OpticConv2d , OpticLinear , 噪声注入器 (PTQ 方案, 保留做推理用)
optic_inference.py	383	光计算推理评估脚本 (Part A)
noise_robustness.py	620	噪声鲁棒性测试 (Part B)

权重文件

文件	大小	来源	精度
baseline_vgg.pth	9.1 MB	Model 1 FP32 训练	97.17%
baseline_vgg_qat.pth	9.1 MB	Model 1 QAT 微调	85.91% (int4)
baseline_vgg_int4.pth	9.1 MB	Model 1 从零 QAT	91.17% (int4)
spacenet_v1.pth	1.0 MB	Model 2 FP32 训练	90.15%
spacenet_v1_qat.pth	1.0 MB	Model 2 QAT 微调	73.63% (int4)
spacenet_v1_int4.pth	1.0 MB	Model 2 从零 QAT	81.20% (int4)
spacenet_v2_distilled.pth	1.0 MB	Model 3 FP32 KD	91.44%
spacenet_v2_qat.pth	1.0 MB	Model 3 QAT 微调	73.22% (int4)
spacenet_v2_int4.pth	1.0 MB	Model 3 从零 KD+QAT	83.26% (int4)
teacher_resnet18.pth	42.7 MB	Model 3 教师	97.83%

文档

文件	用途
OPTIC_QAT_README.md	QAT 技术文档 (中文, 807 行)
EXPERIMENTS.md	本文件: 实验记录
log.md	原始训练日志

10. 结论与下一步

10.1 已完成工作

Phase	方案	最佳结果	判定
0	FP32 基准	97.17% / 90.15% / 91.44%	✓ 基准确立
1	QAT 微调	85.91% / 73.63% / 73.22%	✗ 微调路线失败
2	从零 QAT	91.17% / 81.20% / 83.26%	△ 有效但不够
3	LSQ+混合精度	—	🔄 代码就绪, 待训练

10.2 精度演进总表

	FP32	QAT 微调	从零 QAT	LSQ+混合 (预期)	目标
Model 1	97.17%	85.91%	91.17%	~93-95%	≥95%
Model 2	90.15%	73.63%	81.20%	~84-87%	≥86%
Model 3	91.44%	73.22%	83.26%	~86-89%	≥87%

10.3 待完成

- 运行 Phase 3 训练 (Model 1/2/3 的 `_int4.py`, LSQ+混合精度版本)
- 光计算模拟器验证: 将 int4 权重加载到 `optic_layers.py` + `OpticalEngine`, 用 `optic_inference.py` 测试真实 int4 推理精度
- 噪声鲁棒性评估: 在光模拟器中注入噪声, 用 `noise_robustness.py` 评估
- 与 PTQ 直接量化对比: 同一 FP32 权重, 分别用 PTQ 和 QAT 推理, 量化 QAT 优势

10.4 若 Phase 3 仍不足

备选优化方向:

- 渐进式量化: 前 20% epochs float32 → 30% int8 → 50% int4
- LSQ 输入 scale 校准: 先用一批数据校准 `input_scale`, 再启用 LSQ
- 增加模型宽度: 扩通道补偿 int4 精度损失
- 两阶段 KD+QAT: 先 FP32 KD, 再 QAT 微调 (学习率更低, epochs 更多)